Pandas DataFrame

Agenda

- Introduction to Pandas DataFrame
  - Creating a DataFrame Using Different Methods
  - Accessing the DataFrame
  - Understanding DataFrame Basics
- Introduction to Statistical Operations in Pandas
  - Descriptive Statistics
  - Mean, Median, and Standard Deviation
  - Correlation Analysis

1. Introduction to Pandas DataFrame

A Pandas DataFrame is a two-dimensional, tabular data structure with labeled axes (rows and columns).

It is a primary data structure in the Pandas library, providing a versatile and efficient way to handle and manipulate data in Python.

Series

	Apples
0	3
1	2
2	0
3	1

+

Series

	Oranges
0	0
1	3
2	7
3	2

=

DataFrame

	Apples	Oranges
0	3	0
1	2	3
2	0	7
3	1	2

Key Features:

- **Tabular structure:** The DataFrame is organized as a table with rows and columns, similar to a spreadsheet or SQL table.
- **Labeled axes:** Both rows and columns are labeled, allowing for easy indexing and referencing of data.

- **Heterogeneous data types:** Each column in a DataFrame can contain different types of data, such as integers, floats, strings, or even complex objects.
- **Versatility:** DataFrames can store and handle a wide range of data formats, including CSV, Excel, SQL databases, and more.
- **Data alignment:** Operations on DataFrames are designed to handle missing values gracefully, aligning data based on labels.

1.1 Creating a DataFrame Using Different Methods

Creating a Pandas DataFrame is a fundamental step in data analysis and manipulation.

- Diverse methods are available within Pandas to generate a DataFrame, addressing various data sources and structures.
- Data, whether in Python dictionaries, lists, NumPy arrays, or external files such as CSV and Excel, can be seamlessly transformed into a structured tabular format by Pandas.

```
import pandas as pd

# Creating a DataFrame from a dictionary
data_dict = {'Name': ['Alice', 'Bob', 'Charlie'],
             'Age': [25, 30, 22],
             'Salary': [50000, 60000, 45000]}

df_dict = pd.DataFrame(data_dict)
print(df_dict)

# Creating a DataFrame from lists
data_list = [['Alice', 25, 50000], ['Bob', 30, 60000], ['Charlie', 22, 45000

# Defining column names
columns = ['Name', 'Age', 'Salary']

df_list = pd.DataFrame(data_list, columns=columns)
print(df_list)

# Creating a DataFrame from a NumPy array
import numpy as np
data_array = np.array([['Alice', 25, 50000],
                       ['Bob', 30, 60000],
                       ['Charlie', 22, 45000]])

df_array = pd.DataFrame(data_array, columns=columns)
print(df_array)

# Creating a DataFrame from a CSV file
df_csv = pd.read_csv('HousePrices.csv')
```

```
print(df_csv)

# Creating a DataFrame from an Excel file
df_excel = pd.read_excel('Iris.xlsx')
print(df_excel)
```

```
      Name  Age  Salary
0    Alice   25   50000
1      Bob   30   60000
2  Charlie   22   45000
      Name  Age  Salary
0    Alice   25   50000
1      Bob   30   60000
2  Charlie   22   45000
      Name Age Salary
0    Alice  25  50000
1      Bob  30  60000
2  Charlie  22  45000
                     date         price  bedrooms  bathrooms  sqft_living  `
0     2014-05-02 00:00:00  3.130000e+05       3.0       1.50         1340
1     2014-05-02 00:00:00  2.384000e+06       5.0       2.50         3650
2     2014-05-02 00:00:00  3.420000e+05       3.0       2.00         1930
3     2014-05-02 00:00:00  4.200000e+05       3.0       2.25         2000
4     2014-05-02 00:00:00  5.500000e+05       4.0       2.50         1940
...                   ...           ...       ...        ...          ...
4595  2014-07-09 00:00:00  3.081667e+05       3.0       1.75         1510
4596  2014-07-09 00:00:00  5.343333e+05       3.0       2.50         1460
4597  2014-07-09 00:00:00  4.169042e+05       3.0       2.50         3010
4598  2014-07-10 00:00:00  2.034000e+05       4.0       2.00         2090
4599  2014-07-10 00:00:00  2.206000e+05       3.0       2.50         1490

      sqft_lot  floors  waterfront  view  condition  sqft_above  \
0         7912     1.5           0     0          3        1340
1         9050     2.0           0     4          5        3370
2        11947     1.0           0     0          4        1930
3         8030     1.0           0     0          4        1000
4        10500     1.0           0     0          4        1140
...        ...     ...         ...   ...        ...         ...
4595      6360     1.0           0     0          4        1510
4596      7573     2.0           0     0          3        1460
4597      7014     2.0           0     0          3        3010
4598      6630     1.0           0     0          3        1070
4599      8102     2.0           0     0          4        1490

      sqft_basement  yr_built  yr_renovated                    street  \
0                 0      1955          2005      18810 Densmore Ave N
1               280      1921             0           709 W Blaine St
2                 0      1966             0  26206-26214 143rd Ave SE
3              1000      1963             0           857 170th Pl NE
4               800      1976          1992         9105 170th Ave NE
```

```
...             ...       ...           ...                        ...
4595              0      1954          1979            501 N 143rd St
4596              0      1983          2009          14855 SE 10th Pl
```

1.2 Accessing the DataFrame

Accessing a Pandas DataFrame involves employing various methods for selecting and retrieving data, whether it be specific columns, rows, or individual cells.

- Utilizing square brackets, iloc and loc indexers, and conditions, analysts can navigate and extract the necessary information from the DataFrame for further analysis and manipulation.
- The flexibility of Pandas allows for both label-based and position-based indexing, offering a versatile toolkit for accessing and working with data efficiently.

```
import pandas as pd

# Creating a sample DataFrame
data = {'Column_name': [5, 15, 8],
        'Column1': [10, 20, 30],
        'Column2': [100, 200, 300],
        'Another_column': [25, 35, 45]}

df = pd.DataFrame(data)

# Accessing a single column
column_data = df['Column_name']
print("Single column:")
print(column_data)

# Accessing multiple columns
selected_columns = df[['Column1', 'Column2']]
print("\nMultiple columns:")
print(selected_columns)

# Accessing a specific row by index
row_data = df.iloc[0]
print("\nSpecific row:")
print(row_data)

# Accessing rows based on a condition
filtered_rows = df[df['Column_name'] > 10]
print("\nFiltered rows:")
print(filtered_rows)

# Accessing a single cell by label
value = df.at[0, 'Column_name']
print("\nSingle cell by label:")
print(value)
```

```
# Accessing a single cell by position
value = df.iat[0, 1]  # Row 0, Column 1
print("\nSingle cell by position:")
print(value)

# Accessing data using .loc
selected_data = df.loc[0, 'Column_name']
print("\nData using .loc:")
print(selected_data)

# Conditional access
selected_data = df[df['Column_name'] > 10]['Another_column']
print("\nConditional access:")
print(selected_data)
```

```
Single column:
0     5
1    15
2     8
Name: Column_name, dtype: int64

Multiple columns:
   Column1  Column2
0       10      100
1       20      200
2       30      300

Specific row:
Column_name         5
Column1            10
Column2           100
Another_column     25
Name: 0, dtype: int64

Filtered rows:
   Column_name  Column1  Column2  Another_column
1           15       20      200              35

Single cell by label:
5

Single cell by position:
10

Data using .loc:
5

Conditional access:
```

```
1    35
Name: Another_column, dtype: int64
```

1.3 Understanding DataFrame Basics

- The head() and tail() methods enable users to efficiently preview the initial and final rows of a DataFrame, offering a quick snapshot of its structure and content.
- These functions are invaluable for a preliminary assessment of column names, data types, and potential issues. Additionally, the info() method provides a comprehensive summary, detailing data types, non-null counts, and memory usage, aiding in the identification of missing or inconsistent data.
- The shape attribute, on the other hand, succinctly communicates the dimensions of the DataFrame, encapsulating the number of rows and columns.
- The syntax for some functions is provided below:

Functions	Description	Syntax
head()	Returns the first few rows of the DataFrame	df.head(n)
tail()	Returns the last few rows of the DataFrame	df.tail(n)
info()	Provides a summary of the DataFrame's structure, including column names, data types, and non-null values	df.info()
describe()	Generates descriptive statistics of the DataFrame, such as count, mean, standard deviation, minimum, maximum, and quartiles	df.describe()
shape	Returns the dimensions (number of rows and columns) of the DataFrame	df.shape
columns	Returns the column labels of the DataFrame	df.columns
loc[] or iloc[]	Allows for indexing and slicing of rows and columns based on labels (loc[]) or integer-based positions (iloc[])	df.loc[row_label, column_label]
sort_values()	Sorts the DataFrame based on one or more columns	df.sort_values(by, axis=0, ascending=True, inplace=False)
groupby()	Groups the DataFrame based on one or more columns and allows for aggregating functions such as sum(), mean(), or custom aggregations	df.groupby(by, axis=0, sort=True)
apply()	Applies a function to each element, row, or column of the DataFrame	df.apply(func, axis=0)
merge() or concat()	Combines multiple DataFrames based on common columns (merge()) or concatenates DataFrames along a particular axis (concat())	pd.merge(left, right, on=None, how='inner')
plot()	Creates various types of plots and visualizations based on the DataFrame's data using libraries like Matplotlib or Seaborn	df.plot(x=None, y=None, kind='line', ax=None, ...)
drop()	Removes specified rows or columns from the DataFrame	df.drop(index_label, axis=0, inplace=False)

```
import pandas as pd

# Create a sample DataFrame
data = {'Column_name': [5, 15, 8],
        'Column1': [10, 20, 30],
        'Column2': [100, 200, 300],
        'Another_column': [25, 35, 45]}

df = pd.DataFrame(data)

# Display the first 2 rows
print("First 2 rows:")
print(df.head(2))

# Display the last row
print("\nLast row:")
print(df.tail(1))

# Provide a comprehensive summary of the DataFrame
print("\nDataFrame summary:")
df.info()

# Return a tuple representing the dimensions of the DataFrame (Rows, columns
print("\nDataFrame dimensions:")
print(df.shape)
```

```
First 2 rows:
   Column_name  Column1  Column2  Another_column
0            5       10      100              25
1           15       20      200              35

Last row:
   Column_name  Column1  Column2  Another_column
2            8       30      300              45

DataFrame summary:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3 entries, 0 to 2
Data columns (total 4 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   Column_name     3 non-null      int64
 1   Column1         3 non-null      int64
 2   Column2         3 non-null      int64
 3   Another_column  3 non-null      int64
dtypes: int64(4)
memory usage: 224.0 bytes

DataFrame dimensions:
(3, 4)
```

2. Introduction to Statistical Operations in Pandas

Pandas supports the computation of fundamental measures such as mean and median, along with the exploration of correlations and distribution characteristics.

The following examples illustrate key statistical operations available in Pandas:

2.1 Descriptive Statistics

It offers a snapshot of the dataset's central tendencies and dispersions.

The describe() function provides a quick summary, including mean, standard deviation, and quartile information.

```
import pandas as pd

# Create a sample DataFrame with numeric columns
data = {'Numeric_column1': [5, 15, 8],
        'Numeric_column2': [10, 20, 30],
        'Numeric_column3': [100, 200, 300]}

df = pd.DataFrame(data)

# Display descriptive statistics for numeric columns
print("Descriptive statistics for numeric columns:")
print(df.describe())
```

```
Descriptive statistics for numeric columns:
       Numeric_column1  Numeric_column2  Numeric_column3
count         3.000000              3.0              3.0
mean          9.333333             20.0            200.0
std           5.131601             10.0            100.0
min           5.000000             10.0            100.0
25%           6.500000             15.0            150.0
50%           8.000000             20.0            200.0
75%          11.500000             25.0            250.0
max          15.000000             30.0            300.0
```

2.1 Mean, Median, and Standard Deviation

```
import pandas as pd

# Create a sample DataFrame with numeric columns
data = {'Numeric_column1': [5, 15, 8],
        'Numeric_column2': [10, 20, 30],
        'Numeric_column3': [100, 200, 300]}
```

```
df = pd.DataFrame(data)

# Calculate mean, median, and standard deviation
mean_value = df.mean()
median_value = df.median()
std_deviation = df.std()

print("Mean:\n", mean_value)
print("\nMedian:\n", median_value)
print("\nStandard deviation:\n", std_deviation)
```

```
Mean:
 Numeric_column1      9.333333
Numeric_column2     20.000000
Numeric_column3    200.000000
dtype: float64

Median:
 Numeric_column1      8.0
Numeric_column2     20.0
Numeric_column3    200.0
dtype: float64

Standard deviation:
 Numeric_column1      5.131601
Numeric_column2     10.000000
Numeric_column3    100.000000
dtype: float64
```

2.2 Correlation Analysis

The corr() function generates a correlation matrix, indicating how variables relate to each other.

Values closer to 1 or -1 imply a stronger correlation, while values near 0 suggest a weaker correlation.

```
import pandas as pd

# Create a sample DataFrame with numeric columns
data = {'Numeric_column1': [5, 15, 8],
        'Numeric_column2': [10, 20, 30],
        'Numeric_column3': [100, 200, 300]}

df = pd.DataFrame(data)

# Compute correlation matrix
correlation_matrix = df.corr()
```

```
print("Correlation matrix:\n", correlation_matrix)
```

```
Correlation matrix:
                  Numeric_column1  Numeric_column2  Numeric_column3
Numeric_column1         1.000000         0.292306         0.292306
Numeric_column2         0.292306         1.000000         1.000000
Numeric_column3         0.292306         1.000000         1.000000
```

Value Counts

The value_counts() function tallies the occurrences of unique values in a categorical column, aiding in understanding the distribution of categorical data.

```
import pandas as pd

# Create a sample DataFrame with a category column
data = {'Category': ['A', 'B', 'A', 'C', 'B', 'A', 'C', 'A', 'B', 'C']}
df = pd.DataFrame(data)

# Count occurrences of unique values in the category column
value_counts = df['Category'].value_counts()

print("Value counts:\n", value_counts)
```

```
Value counts:
 Category
A    4
B    3
C    3
Name: count, dtype: int64
```

Assisted Practice

Problem Statement:

Analyze a housing dataset using Pandas DataFrame and statistical operations to understand the basic characteristics of the data and the relationships between different variables.

Steps to Perform:

- Load the housing dataset into a Pandas DataFrame

- Familiarize with the DataFrame basics such as its structure, data types of the columns, and summary statistics
- Calculate descriptive statistics like mean, median, and standard deviation for numerical columns such as **LotArea**, **YearBuilt**, **1stFlrSF**, **2ndFlrSF**, and **SalePrice**
- Determine the correlation between different numerical variables such as **LotArea** and **SalePrice**, **YearBuilt** and **SalePrice**, **1stFlrSF** and **SalePrice**, and **2ndFlrSF** and **SalePrice**
- Count the number of occurrences of each category in categorical variables such as **Neighborhood**, **BldgType**, and **HouseStyle**